

重点回答

SGLang 0.5.14 / CPU 实测环境下的三个核心问题（RadixAttention · page-sized KV cache · vLLM PagedAttention）

Q1 SGLang 的 RadixAttention / RadixCache 解决什么问题？

- 要解决的问题：重复计算与重复存储。**在线服务中大量请求共享前缀——system prompt、多轮对话的历史、few-shot 示例、self-consistency 中同一问题的多次采样。若每个请求独立处理，这些公共前缀的 KV 会被反复 prefill、反复占用内存，是纯粹的浪费。
- 做法：**把 KV cache 组织成一棵 **radix tree（压缩前缀树）**，节点保存 token 区间到 KV 张量的映射；新请求到达时做**最长公共前缀匹配**，命中部分直接复用其 KV，只对未命中的后缀做 prefill；生成的输出也回插树中供后续请求复用。
 - 配套机制：**LRU 淘汰——**先淘汰最久未用的叶子**，使其共同祖先得以长期存活；**引用计数**保证运行中请求的节点不被淘汰；cache-aware scheduling 进一步提升命中率。
 - 关键设计：**不预分配固定大小的缓存池，缓存 token 与运行中请求**共享同一内存池**；当等待请求足够多时，系统会淘汰缓存以换取更大的 batch。
 - 收益：**命中前缀的 prefill 被完全省去 → **TTFT 显著下降**；重复前缀只存一份 → 同等内存下支持更长上下文与更大 batch → 有效吞吐上升。

Q2 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

二者处于**不同层次**：一个是"KV 怎么放"（存储/访存层），一个是"KV 怎么被复用"（调度/计算层）。

机制	所属层次	对应 pipeline 的位置	在流程中起什么作用
page-sized KV cache	存储层 / 访存层 (attention backend)	KV Cache 的分配与寻址： prefill 写入 KV、decode 迭代读取 KV 的那一环	决定 KV 以多大粒度（一个 page 装多少 token）存放与寻址，直接影响碎片率、kernel 访存效率，以及"KV 能否被共享"。
prefix reuse	调度层 / 计算层 (runtime 逻辑)	Scheduler 之后的前缀匹配 → Prefill 增量计算 这一环	决定"哪些 KV 不必重算"：把整段 prefill 变成增量 prefill，收益直接体现为 TTFT 下降。

二者的耦合点——page size 就是那个旋钮：SGLang 的 `--page-size` 控制一个 KV block 内包含多少 token。**page_size = 1** 时前缀复用粒度最细（任意 token 边界都能命中，RadixAttention 论文中原始设计即"每页一个 token"）；**page_size > 1** 时匹配以 page 为粒度，前缀需填满整页才能命中缓存，但对 attention kernel 更友好。因此 page size 是"**复用粒度 ↔ kernel 效率**"之间的显式权衡。

Q3 vLLM 的 PagedAttention 解决什么问题？与 SGLang RadixAttention / paged KV 的联系与差异？

PagedAttention 解决的是 KV cache 的"内存管理"问题。传统实现按序列最大长度**预分配连续显存**，既有内部碎片（预留却未用到）又有外部碎片（空洞无法利用），显存利用率低、batch 上不去。PagedAttention 借鉴操作系统虚拟内存分页：把 KV 切成固定大小的 **block**，用 **block table** 维护"逻辑位置 → 物理 block"的映射，按需分配、非连续存放。效果是显存利用率接近最优，从而支撑更大 batch；并天然支持**块级共享**（copy-on-write），用于并行采样、beam search。

维度	vLLM PagedAttention	SGLang RadixAttention
主要目标	治理显存碎片、提升利用率，扩大 batch size	发现并复用跨请求公共前缀，消除重复 prefill
复用粒度	块哈希精确匹配，需按 block 对齐	radix tree 最长前缀匹配，边界可任意（page_size=1 时按 token）
核心数据结构	block table + 哈希表	radix tree（压缩前缀树），支持节点分裂与共享
回收策略	引用计数 + 块级回收	LRU 淘汰叶子 + 引用计数
所处层次	偏底层：显存布局与 attention kernel 访存	偏上层：缓存复用与调度；底层同样采用 paged KV

联系：① 二者在"KV 的物理布局"上都采用分页思想，**SGLang 的 paged KV 与 vLLM PagedAttention 本质同源**；RadixAttention 论文明确指出其与 paged attention、continuous batching、张量并行**相互兼容**。② vLLM 后续引入 Automatic Prefix Caching（APC），目标与 RadixAttention 一致，说明两条技术路线正在**收敛**。

差异（一句话）：PagedAttention 回答"**KV 怎么放**"——消除碎片、按需分配；RadixAttention 回答"**KV 怎么被复用**"——发现并共享公共前缀。**前者是后者的物理前提，后者在前者之上把"存储共享"进一步转化为"计算省略"**，这正是 TTFT 得以下降的根本原因。

实验侧印证：本次在 CPU 上以 Qwen3-0.6B 启动 SGLang OpenAI-compatible 服务，按 Mooncake FAST'25 trace 采样的请求均**共享同一段长度为 512 token 的公共前缀**（对应 trace 中所有记录共享的 hash_ids[0]），用于触发 RadixCache 前缀复用，并通过 `/metrics` 读取前缀缓存命中率加以验证。